

WEB TEKNOLOJİLERİ ve PROGRAMLAMA

3. Ders: ReactJS



01 Giriş

- Geçmişte, web siteleri ve web uygulamaları sunucuda işlenirdi.
- Bir kullanıcı bir tarayıcıda bir URL'yi ziyaret eder ve bir web sunucusundan bir HTML dosyası ve onunla ilişkili tüm HTML, CSS ve JavaScript dosyalarını ister.
- Bir miktar ağ gecikmesinden sonra, kullanıcı işlenmiş HTML'i tarayıcıda görür ve onunla etkileşime başlar.
- Her ek sayfa geçişi (başka bir URL'yi ziyaret etmek), bu olaylar zincirini yeniden başlatır.
- Geçmişten gelen bu sürümde, esas olarak önemli olan her şey sunucu tarafından yapılırken, istemci yalnızca sayfa görüntüleyerek minimum bir rol oynar.
- Uygulamayı yapılandırmak ve biçimlendirmek için basit HTML ve CSS kullanılırken, etkileşimleri (örneğin, bir açılır menüyü değiştirmek) veya gelişmiş stili mümkün kılmak için JavaScript kullanılırdı.



01 Giriş

- Bu tür işler için popüler bir JavaScript kütüphanesi olan jQuery sık kullanılırdı.
- Buna karşılık, modern JavaScript, odağı sunucudan istemciye kaydırıldı.
- Bir kullanıcı bir URL'yi ziyaret eder, HTML dosyasını ve JavaScript dosyası ister. Bir miktar ağ gecikmesinden sonra, kullanıcı tarayıcıda JavaScript tarafından oluşturulmuş HTML'yi görür ve onunla etkileşime başlar.
- Her ek sayfa geçişi, web sunucusundan daha fazla dosya istemez. Yeni sayfayı oluşturmak için başlangıçta istenen JavaScript'i kullanır.
- Ayrıca, kullanıcı tarafından yapılan her ek etkileşim de istemcide işlenir. Bu modern sürümde sunucu, JavaScript'i tek bir minimum HTML dosyasıyla ağ üzerinden sunar.



01 Giriş

- Daha sonra tüm uygulamayı HTML ile işlemek için istemci tarafındaki dosyalardan bağlantılı JavaScript'i çalıştırılır ve etkileşim için JavaScript kullanılır.
- React tarayıcıdaki her şeyi HTML olarak işlemeyi ve JavaScript ile kullanıcı etkileşimlerini yönetmeyi sağlar.
- React'in yükselişiyle birlikte bileşenler kavramı popüler hale geldi. Her bileşen, görünümünü HTML, CSS ve JavaScript ile tanımlar. Bir bileşen tanımlandıktan sonra, tüm uygulamayı oluşturmak için bileşenler hiyerarşisinde kullanılabilir.
- React'ı kullanmaya başlamadan önce web geliştirme ortamınızı hazırlamanız gerekir.
- **NodeJS**: Javascript kodlarınızı çalıştırmak için gerekli.



01 Kurulum

- **NPM:** Javascript kütüphanelerini yüklemek için gerekli. NodeJS yüklendiğinde otomatik yüklenir.
- VS Code: Web uygulamalarınızı geliştirmenizi, kod yazmanızı sağlayan geliştirme ortamı (IDE).
- CodeSandBox: Online olarak çalışmak isteyenler için ODE.

```
[sinanyuksel@Sinans-MacBook-Pro ~ % node --version  
v16.14.2
```

```
[sinanyuksel@Sinans-MacBook-Pro ~ % npm --version  
8.5.0
```

```
sinanyuksel@Sinans-MacBook-Pro ~ % █
```

01 Kurulum

- Gerekli kütüphaneleri ve araçları kurduktan sonra React uygulamamızı hızlı bir şekilde geliştirmemizi sağlayacak bir şablon üreteceğiz.
- Bunun için React uygulamalarını oluşturmayı kolaylaştıran Vite isimli aracı kullanacağız.
- Aşağıdaki komut ile deneme adında React şablonunu kullanan bir uygulama oluşturacağız. Daha sonra gerekli kütüphaneleri yükleyip, uygulamayı çalıştıracacağız.
 - `npm create vite@latest deneme -- --template react`
 - `cd deneme`
 - `npm install`
 - `npm run dev`

01 Kurulum

```
asimsinanyuksel@Asms-MacBook-Pro Desktop % npm create vite@latest deneme -- --template react
```

```
Need to install the following packages:
```

```
create-vite@8.0.2
```

```
[Ok to proceed? (y) y
```

```
> npx
```

```
> create-vite deneme --template react
```

```
◇ Use rolldown-vite (Experimental)?:
```

```
No
```

```
◇ Install with npm and start now?
```

```
Yes
```

```
◇ Scaffolding project in /Users/asimsinanyuksel/Desktop/deneme...
```

```
◇ Installing dependencies with npm...
```

```
added 199 packages, and audited 200 packages in 24s
```

```
32 packages are looking for funding
```

```
run `npm fund` for details
```

```
found 0 vulnerabilities
```

```
◇ Starting dev server...
```

```
> deneme@0.0.0 dev
```

```
> vite
```

```
VITE v7.1.9 ready in 339 ms
```

```
→ Local: http://localhost:5173/
```

```
→ Network: use --host to expose
```

```
→ press h + enter to show help
```

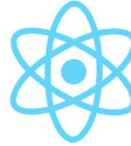


01

Kurulum



localhost:5173



Vite + React

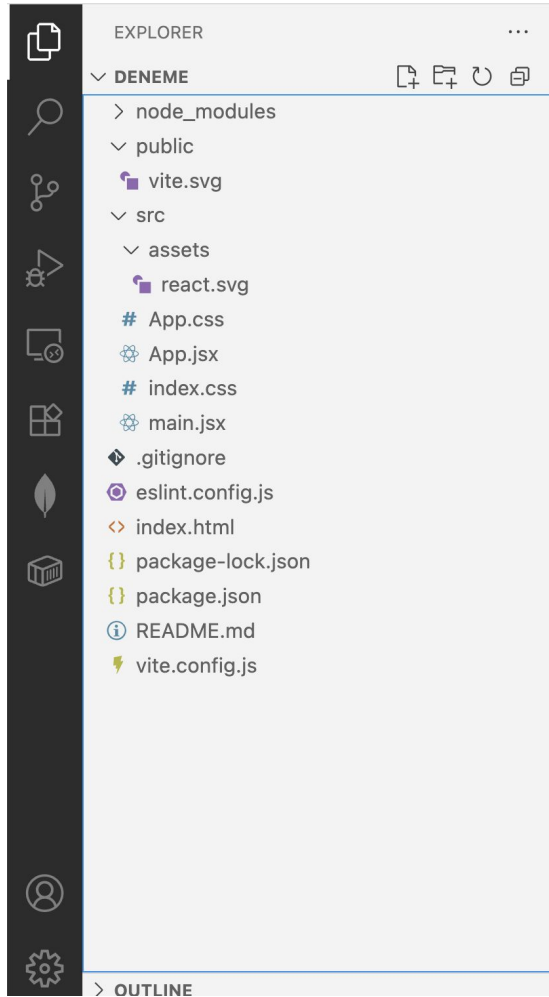
count is 0

Edit src/App.jsx and save to test HMR

Click on the Vite and React logos to learn more



01 Dosya Yapısı



Show All Commands   P

Go to File  P

Open Chat   I

Start Debugging F5

Find in Files   F

01 Dosya Yapısı

- **package.json:** Projenin tüm konfigürasyon ayarlarının, kütüphane/paket bağımlılıklarının yer aldığı dosya
- **node_modules:** yüklenen tüm kütüphaneleri içeren klasör.
- **.gitignore:** git ile gönderilmemesi gereken dosyaların, klasörlerin adlarını içeren dosya.
- **vite.config.js:** Vite'in konfigürasyon ayarlarını içeren dosya.
- **public:** favicon, robots.txt, manifest.json, statik HTML parçaları, sunucuda değiştirilmemesi gereken önceden oluşturulmuş, paketleme/optimizasyon gerektirmeyen dosyalar.
- **index.html:** Proje başlatılırken tarayıcıda görüntülenen HTML'dir. İçerisinde fazla bir içerik yoktur. Ancak, tarayıcıda HTML/CSS çıktısı almak için tüm React kodunun bulunduğu kaynak klasörünüze bağlanan bir komut dosyası etiketi yer alır.
- **src:** React uygulamanızı geliştirmek için ihtiyacınız olan herşeyi içeren klasör.
- **assets:** uygulama içi resimler, ikonlar, fontlar, küçük JSON/CSV; değişikliklerde cache problemi yaşamak istemiyorsanız burayı kullanın.

```
export default defineConfig({  
  plugins: [react()],  
  server: {open: true}  
})
```

Sunucunun otomatik
başlaması için vite.config.js
içerisinde

server: {open: true}

01 Dosya Yapısı

- **App.jsx:** Uygulamanızı kurgulayacağınız, ilerleyen aşamalarda bileşenlerinizi ekleyeceğiniz dosya.
- **main.jsx:** React dünyasına giriş dosyası.
- **index.css, App.css:** Uygulamanıza ve bileşenlerinize stil vermenizi sağlayacak dosyalar.
- **Eslint.config.js:** Projedeki JavaScript/JSX/TS koduna ESLint kurallarını uygular; biçim, hatalar ve kötü pratikleri yakalar. Kod kalitesini yükseltir ve tutarlı stil sağlar. Potansiyel hataları (ör. unused variables, undefined references) erken yakalar.
- package-lock.json dosyası, proje için bağımlılıkların tam sürümlerini (ve alt bağımlılıkların sürümlerini) kilitler. Amaç: her makinede ve CI ortamında aynı bağımlılık ağacının tekrar oluşturulmasını sağlamak — yani tekrarlanabilir (deterministic) kurulum.
- Projenize özgü tüm skriptler package.json dosyasında yer alır. Tüm skriptler “npm run” komutu ile çalıştırılır.
 - **npm run dev:** Uygulamayı tarayıcıda lokal olarak çalıştırır.
 - **npm run build:** Uygulamayı üretim ortamı/production için derler.
 - **npm run preview:** Uygulamayı test etmenizi sağlar. Önce “npm run build” komutunun çalıştırılması gerekir.



01 Dosya Yapısı



```
index.html x
index.html > html > body > div#root
2 <html lang="en">
9   <body>
10   <div id="root"></div>
11   <script type="module" src="...
12   </body>
13 </html>
```

index.html

```
createRoot(document.getElementById('root')).
  <StrictMode>
    <App />
  </StrictMode>,
```

main.jsx

StrictMode: Uygulamanızı bu modda çalıştırdığınızda, React, kodunuzda bazı ek kontroller yapar ve olası performans sorunları, eski kullanım şekilleri veya beklenmeyen davranışlar hakkında sizi uyarır.



01 Linting

- Linting, programlama hatalarını, biçimsel hataları ve hataya neden olabilecek şüpheli yapıları işaretlemek için kullanılan bir statik kod analiz aracıdır.
- Sorunları erkenden belirlemeye yardımcı olduğu ve kodun genel kalitesini iyileştirdiği için yazılım geliştirme döngüsünde önemli bir adımdır.
- Linting ayrıca, bir JavaScript ifadesinin sonunda her zaman noktalı virgül olması veya bulunmaması gibi belirli kodlama standartlarını da uygulayabilir, bu da kodu daha tutarlı ve bakımı kolay hale getirir.
- ESLint en popüler linting aracıdır.

```
"scripts": {  
  "predev": "npm run lint:fix",  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint .",  
  "lint:fix": "eslint . --fix",  
  "preview": "vite preview"  
},
```



01 JSX

- React uygulamalarının temelleri React bileşenleri üzerine kurulur.
- React dünyasında her şey bir bileşen hiyerarşisinden oluşur diyebiliriz.
- İlk oluşturulan uygulamada bileşenler App.jsx dosyasında yer alır.
- Her bir bileşen aslında bir Javascript fonksiyonudur ve PascalCase notasyonunda yazılır.
- Fonksiyonlar aynı zamanda parametre de alabilir. İlerleyen aşamalarda bileşenler arasında bilgi akışını sağlamak için kullanacağız.
- Bileşenler HTML'e benzer bir kod (JSX) döndürür. Bu kod Javascript ve HTML'i birleştirerek dinamik ve interaktif görünümeler elde etmemizi sağlar.
- Fonksiyonun yerine getireceği işleri içerecek kodlar **function** anahtar kelimesi ile return anahtar kelimesi arasına yazılır.
- Bir bileşenin fonksiyonu, bileşenin tarayıcıda her gösterilişinde çalışır.



01 JSX



App.jsx ×

src > App.jsx > ...

```
1 import './App.css';
2 function App() {
3   return (
4     <div>
5       <h1>Merhaba Dünya!</h1>
6     </div>
7   );
8 }
9 export default App;
10
```



01 JSX

- Bir React bileşeninden dönen her şey tarayıcıda gösterilir. Şu ana kadar gördüğümüz örnekte sadece HTML döndürdük.
- Aynı zamanda Javascript içeriği de ekleyebiliriz. HTML ve Javascript'i birleştirerek oluşturulan formata JSX (Javascript XML) denir.

App.jsx > ...

```
1 import "./App.css";
2 function App() {
3   const baslik="Sinan";
4   return (
5     <div>
6       <h1>Merhaba {baslik}!</h1>
7     </div>
8   );
9 }
0 export default App;
```

Merhaba Sinan!



01 JSX

- Kaynak koddaki değişkeni değiştirmek ve tarayıcıya yansıyan değişikliği görmek sadece React'e değil, komut satırında uygulamamızı başlattığımızda altta yatan geliştirme sunucusuna da bağlıdır.
- Dosyalarımızdan biri değiştiğinde, geliştirme sunucusu bunu fark eder ve etkilenen tüm dosyaları tarayıcı için yeniden yükler.
- React ile geliştirme sunucusu arasındaki bu davranışı mümkün kılan köprü, React tarafında **React Fast Refresh** (Önceki adı: React Hot Loader) ve geliştirme sunucusu tarafında **Hot Module Replacement** olarak adlandırılır.
- Bir önceki örneğimizi geliştirerek bir input field ve label kullanarak basit bir arama kutusu yapalım.
- JSX ve HTML arasındaki temel farkları görmeye çalışalım.



01 JSX



App.jsx ×

src > App.jsx > ...

```
1 import './App.css';
2 function App() {
3   const baslik = "Sinan";
4   return (
5     <div>
6       <h1>Merhaba {baslik}!</h1>
7       <label htmlFor="arama">Ara: </label>
8       <input id="arama" type="text" />
9     </div>
10  );
11 }
12 export default App;
13
```



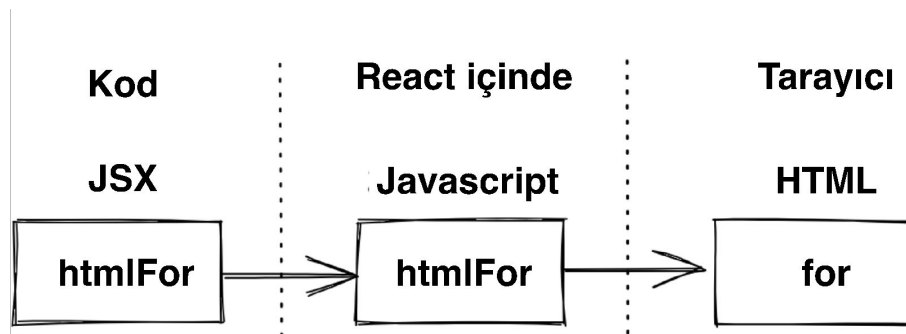
Merhaba Sinan!

Ara:



01 JSX

- id ve type ifadelerine HTML'den aşına olabilirsiniz. **htmlFor** ifadesi sizin için yeni olabilir.
- htmlFor HTML'deki for ifadesine denk gelmektedir. JSX buna benzer bir çok özelliği kendi ifadeleriyle değiştirir. Değiştirdiği ifadelerde camelCase notasyonunu kullanır.
- class yerine className; onclick yerine onClick vb. gibi.
- JSX içinde HTML kullandığınızda, React tüm HTML özelliklerini Javascript'e dönüştürür. Ancak HTML elementleri tarayıcı tarafında gösterilirken tekrar orijinal versiyonlarına dönüştürülür.



01 JSX

- Bir önceki örneği değiştirerek basit değişkenler yerine daha karmaşık olan Javascript nesneleriyle çalışalım.
- baslik isimli basit değişken yerine karsilama isimli bir nesne ve bu nesnenin içinde selamlama ve baslik alt değişkenleri tanımlayalım.

```
App.jsx > ...
1  import "./App.css";
2  function App() {
3    const karsilama = {
4      "selamlama": "Merhaba",
5      "baslik": "Sinan!"
6    }
7    return (
8      <div>
9        <h1>{karsilama.selamlama} {karsilama.baslik}</h1>
10       <label htmlFor="arama">Ara: </label>
11       <input id="arama" type="text" />
12     </div>
13   );
14 }
15 export default App;
```

01 JSX

- Süslü parantezler içindeki her şey Javascript'e dönüştürülecektir. Değişken tanımlamanın yanında bir fonksiyon da tanımlayabilir ve bu fonksiyonu süslü parantezler içinde kullanarak çağırabilirsiniz.

App.jsx > ...

```
import "./App.css";
function App() {
  function karsila(selamlama,baslik) {
    return selamlama + " " + baslik;
  }
  return (
    <div>
      <h1>{karsila("Merhaba", "Sinan!")}</h1>
      <label htmlFor="arama">Ara: </label>
      <input id="arama" type="text" />
    </div>
  );
}
export default App;
```



01 Listeler

- React'te, dizinin/array yerleşik `map()` metodu kullanılarak listeleme işlemleri yapılır.

```
const yaziListesi = [
  {
    baslik: "React Öğreniyorum",
    yazar: "Sinan Yüksel",
    yorum_sayisi: 3,
    puan: 4,
    id: 0,
  },
  {
    baslik: "Web Teknolojileri ve Programlama",
    yazar: "Asım Yüksel",
    yorum_sayisi: 2,
    puan: 5,
    id: 1,
  },
];
```

```
function App() {
  return (
    <div>
      <h1>Yazılar</h1>
      <label htmlFor="arama">Ara: </label>
      <input id="arama" type="text" />
      <hr />
      <ul>
        {yaziListesi.map(function (yazi) {
          return <li>{yazi.baslik}</li>;
        })}
      </ul>
    </div>
  );
}
```

! ▶ Warning: Each child in a list should have a unique "key" prop.

01 Listeler

- Listedeki her bir React elemanının benzersiz bir key alanı olmalıdır.
- Örnek: <https://codesandbox.io/s/kind-frost-woyu4n>

```
function App() {  
  return (  
    <div>  
      <h1>Yazılar</h1>  
      <label htmlFor="arama">Ara: </label>  
      <input id="arama" type="text" />  
      <hr />  
      <ul>  
        {yaziListesi.map(function (yazi) {  
          return <li key={yazi.id}>{yazi.baslik}</li>;  
        })}  
      </ul>  
    </div>  
  );  
}
```

Yazılar

Ara:

- React Öğreniyorum
- Web Teknolojileri ve Programlama

01 Bileşen Oluşturma

- Bileşenler, her React uygulamasının temelidir.
- Uygulamanız büyüdükçe, yönetmeniz gereken daha fazla bileşen olacaktır.
- Her bileşenin bir işlevi vardır. Şimdiye kadar yalnızca **App** bileşenini kullandık. Bileşenlerin uygulamanızın boyutuna göre ölçeklenmesi gerektiğinden, bunun sonu pek iyi olmayacaktır.
- Bu nedenle, bir bileşeni zamanla daha büyük ve daha karmaşık hale getirmek yerine, bir bileşeni birden çok bileşene bölmek daha yönetilebilir bir yapı sunacaktır.
- Bu nedenle, App bileşeninden listeleme işlevleri çıkarıp, yeni bir Liste bileşeni oluşturacak ve listeleme görevini bu bileşene vereceğiz.

01 Bileşen Oluşturma



```
function Liste() {
  return (
    <div>
      <ul>
        {yaziListesi.map(function (yazi) {
          return (
            <li key={yazi.id}>
              <span>
                <a href={yazi.url}>{yazi.baslik}</a>,
              </span>
              <span><b>Yazar:</b> {yazi.yazar}, </span>
              <span><b>Yorum Sayısı:</b> {yazi.yorum_sayisi}, </span>
              <span><b>Puan:</b> {yazi.puan}</span>
            </li>
          );
        })}{ " " }
      </ul>
    </div>
  );
}
```

```
function App() {
  return (
    <div>
      <h1>Yazılar</h1>
      <label htmlFor="arama">Ara: </label>
      <input id="arama" type="text" />
      <hr />
      <Liste/>
    </div>
  );
}
```

01 Bileşen Oluşturma



- İlk bileşeninizi oluşturdunuz.
- Bu örneği göz önünde bulundurarak, bileşenlerin daha büyük bir React projesini nasıl iyileştirdiğini, anlamlı görevleri nasıl özetlediğini görebiliriz.
- Bir bileşen oluşturmak, bir React geliştiricisi olarak çok sık gerçekleştireceğiniz bir görevdir.
- Çünkü bir bileşenin boyutu ve karmaşıklığı her zaman büyüyecektir. Büyük bileşenleri küçük bileşenlere parçalayarak projenizi daha esnek, daha yönetilebilir yapabilirsiniz.
- Şimdi arama işlevini App bileşeninden ayıralım ve kendi bileşenini oluşturalım.
- Sonuçta App, Liste ve Arama bileşenlerimiz olacak.



01 Bileşen Oluşturma



```
function App() {  
  return (  
    <div>  
      <h1>Yazılar</h1>  
      <Arama/>  
      <hr />  
      <Liste/>  
    </div>  
  );  
}
```

```
function Arama() {  
  return (  
    <div>  
      <label htmlFor="arama">Ara: </label>  
      <input id="arama" type="text" />  
      <hr />  
    </div>  
  );  
}
```

Örnek: <https://codesandbox.io/s/upbeat-ace-fy4yjs>

01 Bileşen Oluşturma

- React uygulamalarının bileşen hiyerarşileri vardır (component tree).
- Genellikle, bileşenlerin, ağacı kapsayan en üstteki bir giriş noktası bileşeni (App gibi) vardır.
- App, Liste ve Arama'nın ana bileşenidir. Bu nedenle Liste ve Arama, App bileşeninin alt bileşenleri (child) ve birbirinin kardeş (sibling) bileşenleridir.
- Bir bileşen ağacında her zaman bir kök bileşen (App.jsx) vardır ve diğer bileşenleri oluşturmayan bileşenlere yaprak/leaf bileşenler (Liste, Arama) denir.
- Tüm bileşenlerin sıfır, bir veya birçok alt bileşeni olabilir.
- Bir bileşen oluşturduğunuzda, artık onu JSX'in herhangi bir yerinde element olarak kullanabilirsiniz. Element bir bileşenin örneğidir ve istediğiniz kadar örnek oluşturabilirsiniz.



01 Bileşen Oluşturma

- Bileşen tanımlarken standart tanımlama dışında **arrow** notasyonu da kullanılabilir.
- Bu noktada function ifadesi kaldırılır. Her bir bileşenin başına const ifadesi getirilir.
- Aynı işlem callback/geri çağırım metotları için de yapılabilir.
- Bu tanımlama isteğe bağlı bir işlemdir. Uygulamalarda ikisinden birini tercih ederek yola devam edebilirsiniz. Ekip olarak çalıştığınız projelerde ise herkesin aynı notasyonu kullanması daha faydalı olacaktır.

```
function App() {
```

```
function Arama() {
```

```
function Liste() {
```

```
{yaziListesi.map(function (yazi) {
```

```
const App = () => {
```

```
const Arama = () => {
```

```
const Liste = () => {
```

```
{yaziListesi.map((yazi) => {
```

01 React DOM

- Bir bileşeni nasıl oluşturacağımızı ve kullanacağımızı gördük. Ancak App bileşeninin örneğini hiçbir yerde oluşturmadık.
- Bir yerlerde bu bileşenin bir örneğinin oluşturulması gerekir. Aksi halde hiyerarşideki hiçbir bileşen tarayıcıda görünmeyecektir.
- main.jsx dosyasını açtığınızda App bileşenin burada oluşturulduğunu göreceksiniz.
- Dosyanın başında içe aktarılan iki kütüphane vardır: react ve react-dom.
- react, bir React geliştiricisinin günlük işleri için kullanılırken, react-dom, React'i yerel HTML dünyasına bağlamak için genellikle bir React uygulamasında bir kez kullanılır.
- createRoot metodu React uygulamasını başlatan HTML elementini parametre olarak alır.



01 React DOM

- DOM: Tarayıcıda çalışan her web sayfasının ağaç şeklindeki temsidir.
- HTML kodları → Tarayıcı → DOM yapısı
- DOM, JavaScript ile dinamik olarak değiştirilebilir.
- React, kullanıcı arayüzlerini tanımlamak için bir kütüphanedir.
- React tek başına DOM'a dokunmaz.
- React DOM, React bileşenlerini alır ve gerçek DOM'a yansıtır.
 - `const element = <h1>Merhaba React!</h1>;`
 - `ReactDOM.render(element, document.getElementById('root'));`
- React, "Virtual DOM" adında bir hafif kopya oluşturur.
- Her değişiklikte tüm DOM'u değil, sadece değişen kısmı günceller.
- Bu yüzden React çok hızlı çalışır. Örnek.
 - Gerçek DOM → Dev bir ansiklopedi
 - Sanal DOM → Hafızadaki notlar
 - Sadece farkları yazarsın, kitabı baştan yazmazsın.



01 React DOM

- React, bileşeni render eder ve sanal DOM oluşturur.
- Bir değişiklik olduğunda yeni sanal DOM'u oluşturur.
- Diff algoritması iki sanal DOM'u karşılaştırır.
- Farklı olan düğümler sadece gerçek DOM'da güncellenir.
- Sonuç: Minimum yeniden çizim → Maksimum performans.
- React bileşen ağacını Fiber yapısına dönüştürür.
- Bu yapı "işleri parçalara böler" ve daha verimli render sağlar.
- Tarayıcı boşa kaldığında güncelleme yapılır → non-blocking UI.

```
function Counter() {  
  const [count, setCount] = React.useState(0);  
  return (  
    <div>  
      <p>Sayaç: {count}</p>  
      <button onClick={() => setCount(count + 1)}>Artır</button>  
    </div>  
  );  
}  
createRoot(document.getElementById('root')).render(<Counter />);
```

DOM'da sadece
<p> etiketi
güncellenir, tüm
sayfa değil!

01 React DOM

- Eskiden React, sanal DOM'daki değişiklikleri hesaplayıp tek seferde DOM'a uygular, bu sırada JavaScript işlemi bitene kadar tarayıcıyı bloke ederdi. Yani sayfa bir anda donabilir, kullanıcı etkileşimleri gecikebilirdi.
- React ekibi bunu çözmek için Fiber Architecture adında yeni bir sistem geliştirdi (React 16'dan itibaren).
- Fiber, React bileşen ağacını "fiber" denen küçük birimlere böler.
- Her fiber, bir bileşeni ve onun durumunu temsil eder:
- hangi props'lar var, hangi state değişmiş, hangi DOM düğümüyle ilişkili, vb.
- Bu sistemin amacı:
- Büyük bir işi küçük parçalar halinde, kesintisiz ve kontrol edilebilir biçimde işlemek.



01 React DOM



- **Parçalı (incremental) render:**

- React, bileşenleri tek tek işler, her birini kısa bir “iş paketi” olarak ele alır.
- Eğer tarayıcıya acil bir iş (örneğin kullanıcı tıklaması) gelirse,
- React o anki işi yarıda kesebilir ve sonra kaldığı yerden devam eder.
- Bu, non-blocking yani kesintisiz kullanıcı deneyimi sağlar.

- **Prioritization (önceliklendirme):**

- Her işin bir önceliği vardır:
- Kullanıcı etkileşimi → yüksek öncelik
- Arka plandaki liste hesaplaması → düşük öncelik
- React bu öncelikleri göz önüne alarak render sırasını ayarlar.

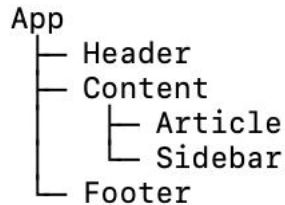
- **Reconciliation (uzlaştırma):**

- Fiber yapısı, sanal DOM farklarını hesaplarken bu küçük fiber düğümlerini kullanır.
- Böylece, sadece değişen kısımlar DOM’a yansıtılır.



01 React DOM

- Diyelim ki React uygulaman bir ağaç:



- Eskiden React bu ağacı tek hamlede işlerdi.
- Fiber ile:
- Her bir bileşen kendi “fiber” nesnesi olur.
- React bu fiber’ları sırayla işler,
- Gerekirse tarayıcıya “bir nefes al” der,
- Sonra kaldığı yerden devam eder.
- Sonuç: akıcı, hızlı, kesintisiz render.



01 Virtual DOM

- Virtual DOM, React'in hafızada tuttuğu bir DOM kopyasıdır.
- Bu, JavaScript nesneleriyle temsil edilir — yani tarayıcıda değil, RAM'de, React'in kendi içinde yaşar.
- Bir bileşen render edildiğinde:
 - React bir Virtual DOM ağacı oluşturur.
 - Bu, “ekranda ne olması gerektiğini” tarif eder.
 - Tarayıcıya dokunmadan önce tüm hesaplamalar burada yapılır.
- Yani sanal DOM gerçek DOM'un temsilidir, ama tarayıcıya yazılmamış halidir.
- Virtual DOM:
 - Tarayıcıda ama tarayıcının çekirdeğinde değil,
 - React'in hafızasında (JavaScript runtime içinde) bulunur.



01 Virtual DOM



- **Render aşaması:**
 - React bileşenleri çalıştırır, yeni bir Virtual DOM ağacı oluşturur.
- **Diffing aşaması:**
 - Yeni Virtual DOM ile eski Virtual DOM karşılaştırılır.
 - Farklı olan düğümler bulunur.
- **Commit aşaması:**
 - Sadece değişen düğümler gerçek DOM'a yazılır.
 - Böylece tarayıcı minimum iş yapar.

```
function App() {  
  const [count, setCount] = React.useState(0);  
  return <h1>{count}</h1>;  
}
```

- İlk render: Virtual DOM = <h1>0</h1>
- Butona basınca: Virtual DOM = <h1>1</h1>
- React karşılaştırır: sadece "text node" değişmiş.
- → Gerçek DOM'da sadece o kısım güncellenir.



01 Virtual DOM

Özellik	Gerçek DOM	Sanal DOM
Nerede bulunur	Tarayıcı motorunda	React'in hafızasında
Güncelleme maliyeti	Yüksek	Düşük
Güncelleme şekli	Doğrudan	Fark bazlı (diff)
Kullanım amacı	Sayfayı gösterir	Değişiklikleri hesaplar

Kavram	Ne yapar	Nerede durur
Virtual DOM	UI'nin "önceki" ve "şimdiki" halini karşılaştırır (diffing)	React'in belleğinde
Fiber	Bu karşılaştırma ve güncelleme işlemini küçük parçalara ayırır, önceliklendirir	React'in render motorunda

Virtual DOM, "gölgede çalışan ikinci bir dünya" gibidir:

Tarayıcıya gitmeden önce tüm kararlar burada alınır.

Virtual DOM "neyin değiştiğini" bulur,
Fiber "bu değişiklikleri ne zaman ve hangi sırayla" uygulayacağını yönetir.



- 1. Bileşen çalışır → Virtual DOM oluşturulur.
 - React, her bileşeni çalıştırarak yeni bir “virtual DOM ağacı” çıkarır.
 - Bu, “ekranda ne görünmeli?” sorusunun cevabıdır.
- 2. Fiber yapısı → Bu ağacı yönetir.
 - Virtual DOM’un her düğümü aslında bir Fiber nesnesi ile temsil edilir.
 - Fiber, bu düğümün: Props’unu, State’ini, Çocuklarını, Önceliğini, Hangi DOM elemanına denk geldiğini saklar.
- 3. React iki Virtual DOM ağacını karşılaştırır (diffing).
 - Bu işlemi Fiber düğümleri üzerinden yapar.
 - Her Fiber bir “iş paketi” gibidir — değişmişse güncellenir, değilse geçilir.
- 4. Fiber güncellemeleri planlar.
 - Örneğin, “şu bileşen çok önemli, hemen güncelle; şu diğeri bekleyebilir” der.
 - Böylece React uzun render işlemlerini kesip sonra devam edebilir.